



**UGANDA CHRISTIAN
UNIVERSITY**

A Centre of Excellence in the Heart of Africa

**UGANDA CHRISTIAN UNIVERSITY
FACULTY OF ENGINEERING, DESIGN AND TECHNOLOGY
DEPARTMENT OF COMPUTING AND TECHNOLOGY**

Course Unit: AI Model Deployment and Scalability (CSC3119)

Track: Artificial Intelligence and Robotics

Assignment 4

Approach Report

Predicting PM2.5 Air Quality Concentrations from AirQo Sensor Data

Group Members

No.	Name	Registration No.
1	Charis Opol	S24B23/003
2	Kirabo Esther Kebirungi	S24B23/045
3	Ansasiire Nehum Naamaye	S24B23/018
4	Norah-Angel Nakaye	M24B23/008
5	Mwonge Andrea	S24B23/070
6	Cole Rodney Migisha	S24B23/006
7	Ssendagire Abubaker	S24B23/012
8	Shane Munanura	M24B23/031
9	Mirembe Peace Mercy	M24B23/005
10	Tony Blair	S24B23/024
11	Musoke Nestroy	S24B23/069

September 15, 2026

This report documents the approach agreed during the team's planning meeting for Assignment 4. Where the meeting settled on a specific method, that method is stated as the approach the pipeline will follow; open questions raised in the meeting are noted as decisions still to be finalised during individual group implementation.

1. Data Preparation

1.1. Null value handling

The dataset defines a value of 0 or an empty field as null for temp, humidity, pm2_5_calibrated_value, pm10_calibrated_value, latitude and longitude, since a genuine zero reading is not physically meaningful for these fields. The team agreed these nulls should not be blanket-dropped, since sensors going offline for stretches of time is a normal feature of a low-cost sensor network and should be represented in the pipeline rather than eliminated.

Two further points were raised and agreed as design constraints rather than one-off fixes: (1) because the model will run on incoming batch/API data that may itself contain nulls, the model family chosen must tolerate missing values gracefully rather than fail on them, which favours tree-based models for the core prediction path; and (2) the pipeline should be able to store a prior model checkpoint and roll back to it if a retrain on new data produces materially worse validation performance, protecting the deployed pipeline from a bad batch of readings.

1.2. Feature selection

Following Section 2 of the brief, temperature and humidity are retained as predictors for PM2.5 (they affect boundary-layer height and secondary aerosol formation) but are excluded from any PM10 modelling, since PM10 is mechanically generated dust and is largely insensitive to them. Site identifiers, coordinates, and time-derived features (hour of day, day of week, season) are retained; administrative fields such as sample_rate and network_note are used only to construct the offline/missing flag and are not fed to the model as predictors.

1.3. Hourly resampling and data types

Each sensor is reindexed to a complete hourly timestamp series spanning January to July 2026, so that offline hours appear as missing rows rather than absent ones; this is required for consistent windowing and lag-feature construction later in the pipeline. Timestamps are enforced as datetime, all measurement fields as float, and identifiers (site_id, sensor_name, device_name) as consistent string/categorical types across the full dataset.

1.4. Field-level handling summary

Field	Approach agreed	Rationale
PM2.5 / PM10 / temp / humidity	Treat 0 or empty as missing (not a genuine reading). Flag as sensor-offline rather than dropped; interpolate short gaps, otherwise leave as missing for the resampled index.	Zero is physically implausible for these fields and offline stretches are expected, not anomalies.
Latitude / Longitude	Median imputation per site as the default convergence point; where the nearest site also has (0,0), fall back to the site's own historical median rather than borrowing from another site.	Median gives a stable, outlier-resistant fixed point for a site whose coordinates should not drift.

2. Exploratory Data Analysis

EDA is carried out on roughly five Kampala-based sensors, per the brief. This covers distribution, time-series and box plots of PM2.5 with outliers flagged and their treatment logged; a calendar plot of the cleaned, unscaled PM2.5 values colour-coded by the AQI bands in Section 4 of the brief; and time-series plots with the same AQI bands rendered as shaded background bands so periods of unhealthy air are visible at a glance.

A Z-score transform is applied to PM2.5 across all sensors; because this centres and scales the series to zero mean and unit variance, a value's magnitude is interpreted as how many standard deviations that reading sits from the sensor's own mean, rather than as a $\mu\text{g}/\text{m}^3$ concentration. This transformed series is the modelling target, with results inverse-transformed back to $\mu\text{g}/\text{m}^3$ for reporting. Correlation between temperature, humidity and PM2.5 is computed for the same five Kampala sensors to confirm the meteorological relationship described in the brief before those features are relied on downstream.

3. Model Implementation

Since this is a prediction task on a continuous target, the team confirmed regression models are appropriate. Five models plus at least one ensemble are trained, drawn from the three groups below (core linear, tree-based/ensemble, advanced/non-linear); each sub-team selects models from its assigned group and tunes the hyperparameters set out in the regression hyperparameter guide, documenting why each tuned parameter matters for that model.

- **Core:** Linear, Polynomial, Ridge, Lasso, ElasticNet. Tuning centres on alpha (and l1_ratio for ElasticNet) or polynomial degree, balancing bias against overfitting.
- **Tree-based/ensemble:** Decision Tree, Random Forest, Gradient Boosting, XGBoost/LightGBM/CatBoost. Tuning centres on depth, n_estimators, learning_rate and subsampling, with early stopping used for the boosting frameworks.
- **Advanced/non-linear:** SVR, KNN, Quantile Regression, Neural Network. Tuning centres on C/epsilon/gamma, n_neighbors, the target quantile, and network depth/dropout respectively.

Evaluation uses RMSE and MAE (interpretable in the same units as the Z-scored target, and RMSE in particular penalises large misses, which matters for catching unhealthy-air spikes) alongside R^2 to summarise overall fit; MAE is favoured for a plain-language readout when reporting to non-technical stakeholders. The best-performing model on the validation metrics is identified and the trained artifact is serialised for use in forecasting and deployment.

4. Forecasting and Visualization

The saved model produces daily PM2.5 predictions for all sensors across the first two full weeks of August 2026, and hourly predictions for the first full day of August 2026. Both forecasts are inverse Z-score transformed back to $\mu\text{g}/\text{m}^3$ before interpretation. The daily forecasts are visualised on a calendar plot and a time-series plot for roughly five sensors chosen across different zones, with the Section 4 AQI legend embedded so safe versus unsafe periods are immediately readable.

5. Model Monitoring with Evidently AI

Evidently AI is integrated to track two distinct signals: data drift, which flags when incoming feature distributions (e.g. temperature, humidity, PM2.5 itself) shift away from the training distribution, and model drift, which flags when prediction error grows over time even if inputs look stable. Both matter for this deployment because sensor behaviour and seasonal pollution patterns change over the six-month window, and each metric Evidently reports is explained in the full report in terms of what it implies for this specific PM2.5 pipeline rather than generically.

6. Deployment on RunPod

The team agreed resource needs will be measured empirically rather than estimated on paper, specifically by testing the prediction step only, since training consumes substantially more memory than inference and would overstate what the deployed endpoint actually needs. A test run records four figures: GPU memory used, RAM used by the data pipeline, per-prediction latency, and the combined size of the model plus its inputs/outputs. The team will also confirm whether the forecast is produced in a single multi-step pass or generated one step at a time and fed back in (autoregressively), since the latter is materially slower for the two-week daily horizon and affects which GPU tier is worth choosing.

These measurements are then used to select the cheapest RunPod GPU able to run the pipeline within the \$5 budget. The deployment is configured as a Pod rather than a serverless endpoint, since the pipeline is still being tested, using a PyTorch template. Data is imported, the full pipeline is run on the Pod, and the resulting metrics and runtime logs are saved as evidence the pipeline ran successfully within budget. Per the brief, the pipeline is run cleanly end-to-end locally first, and if Evidently AI errors specifically on RunPod, that block is commented out so the rest of the pipeline still completes.